

AI 활용 기술 문서

Unity 기반 방치형 RPG · AI 코딩 에이전트를 활용한 개발 구조와 지시 설계

프로젝트 Inflearn_Seed / IdleProject
엔진 Unity 6000.3.13f1 (Universal Render Pipeline 17.3.0)
저장소 github.com/PinousCEO/Inflearn_Seed
작성일 2026-08-10

1. 문서 개요

이 문서는 본 프로젝트에서 AI를 어떤 구조로 개발 과정에 결합했는지, 그리고 그 AI에게 어떤 지시를 어떤 방식으로 주었는지를 기술한다. 마지막 장에는 사용한 외부 에셋과 오픈소스의 출처를 정리했다.

본 프로젝트에서 AI는 "코드를 대신 타이핑해 주는 도구"가 아니라, **Unity 에디터에 직접 접속해 자신이 만든 결과를 검증하는 실행 주체**로 사용되었다. 아래 3장의 파이프라인 구조가 이 문서의 핵심이다.

85

C# 스크립트

22,423

코드 라인

20

에디터 자동화 도구

61

절차적 합성 효과음

2. AI 활용 개요

2.1 사용 도구

도구	역할	비고
Claude Code (Anthropic)	주 개발 에이전트. 코드 작성·리팩터링·디버깅·에셋 데이터 생성	터미널에서 저장소에 직접 접근. 프로젝트 로컬 설정은 <code>IdleProject/.claude/settings.local.json</code> 에 누적되어 있으며, 6개 이상의 개별 작업 세션 기록이 남아 있다.
Unity MCP (<code>com.unity.ai.assistant</code> 2.17.0-pre.1)	AI ↔ Unity 에디터 연결 통로	AI가 에디터 안에서 C# 코드를 컴파일·실행하고, 콘솔 로그와 카메라 렌더 결과를 회수할 수 있게 하는 브리지
보조 CLI 에이전트	일부 작업 구간에서 병행 사용	저장소 stash에 <code>codex-before-pull-2026-08-05</code> 기록이 남아 있음

2.2 AI가 담당한 영역과 사람이 담당한 영역

AI가 주로 수행	사람(개발자)이 수행
- 게임 시스템 C# 구현 전반 - 에디터 자동화 도구 작성	- 게임 기획·재미 판단·최종 의사결정 3D/2D 에셋 선정 및 씬 아트 배치

- 효과음 절차적 합성 알고리즘
- ScriptableObject 데이터 에셋 생성
- 3개 국어 로컬라이제이션 표 구성
- 버그 원인 추적 및 수치 밸런싱 시뮬레이션

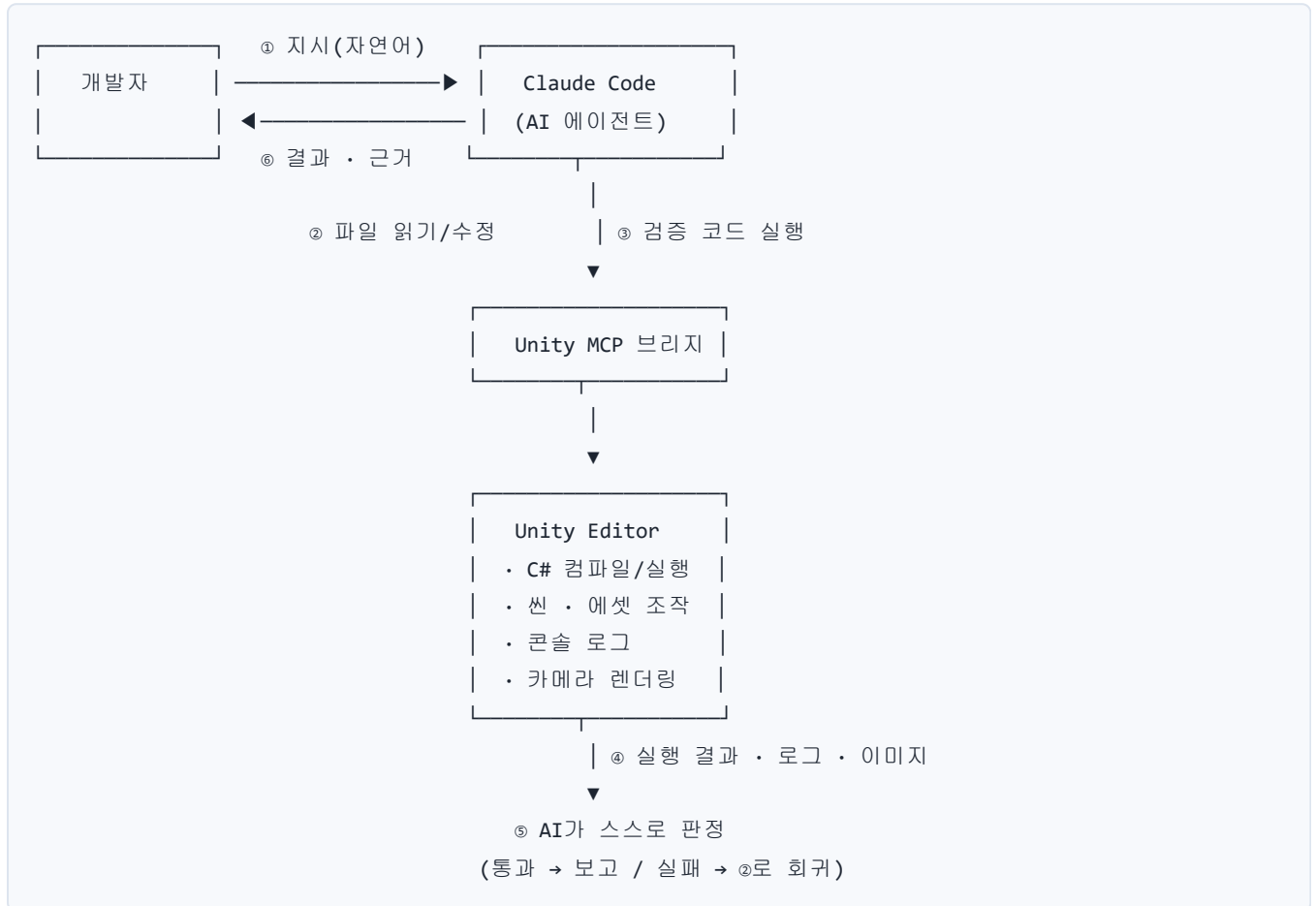
- 애니메이션 클립 확보 및 임포트
- 실제 플레이 테스트와 체감 피드백
- AI 결과물에 대한 검수·반려·재지시

기록의 한계 안내. 본 문서는 저장소에 실제로 남아 있는 증거(설정 파일, 커밋 이력, 코드 및 주석, 패키지 매니페스트)에 근거해 작성했다. 개발 초기 일부 구간의 도구 사용 내역은 로그로 남아 있지 않아, 확인 가능한 범위로 한정해 기술했다.

3. 개발 파이프라인 구조

3.1 AI ↔ Unity 직결 구조

일반적인 AI 코딩은 *텍스트 파일을 고치는 것*에서 끝나고, 그 결과가 실제로 동작하는지는 사람이 에디터를 열어 확인해야 한다. 본 프로젝트는 그 확인 단계까지 AI가 스스로 수행하도록 **Unity MCP 브리지**를 개발 루프에 포함시켰다.



이 구조에서 AI가 실제로 수행할 수 있는 검증은 다음과 같다.

검증 수단	확인할 수 있는 것
에디터 내 C# 실행	컴파일 성공 여부, 에셋.씬의 실제 직렬화 값, 참조 연결 상태
콘솔 로그 회수	런타임 예외, 경고, 누락된 참조
카메라 렌더 캡처	화면에 실제로 보이는 결과(색상·배치·연출)
씬 오브젝트 순회	인스펙터 연결 누락, 계층 구조 이상

3.2 검증 루프의 실제 효과

이 구조 덕분에 "코드는 맞는데 에디터 설정이 틀린" 유형의 결함을 AI가 스스로 잡아낼 수 있었다. 이 유형은 코드만 읽어서는 절대 발견되지 않는다. 대표적인 두 사례는 6장에서 상세히 다룬다.

3.3 코드 생성이 아니라 "생성기 생성"

UI 화면을 AI가 만들 때, 완성된 씬 파일을 직접 편집하는 대신 **그 화면을 만들어 내는 에디터 스크립트를 작성**하는 방식을 택했다. Unity의 `.unity / .prefab` 은 거대한 YAML이라 AI가 직접 편집하면 파손 위험이 크고 재현이 어렵기 때문이다.

에디터 자동화 도구	생성 대상
<code>ShopPanelBuilder</code>	상점 화면 전체 계층
<code>MailboxPanelBuilder</code>	우편함 화면
<code>SettingsPanelBuilder</code>	설정 화면
<code>EquipmentPanelBuilder</code>	장비 화면 및 스크롤바
<code>PassiveTreePrefabBuilder</code>	패시브 스킬 트리 노드/연결선
<code>BottomNavigationSetup</code>	하단 내비게이션 바
<code>LootRarityPrefabBuilder</code>	등급별 전리품 연출 프리팹
<code>CharacterAnimatorSkillSetup</code>	애니메이터 스킬 상태·전이 배선
<code>LocalizationBinder</code>	씬 텍스트에 번역 키 일괄 부착
<code>EquipmentItemDataBuilder</code> 외	아이템·스킬·스탯 데이터 예셋 일괄 생성

총 20개의 에디터 도구(약 5,966줄)가 이 목적으로 작성되었다. 결과적으로 UI 구성이 **재실행 가능한 코드로 문서화**되어, 수정이 필요하면 도구를 고쳐 다시 돌리면 된다.

4. AI 활용 주요 사례

4.1 효과음 절차적 합성 — 음원 파일 없는 사운드

본 게임의 효과음은 구매하거나 녹음한 음원 파일이 아니다. `SfxSynth`가 런타임에 파형을 직접 계산해 `AudioClip`을 만든다. 저장소에 포함된 오디오 파일은 BGM 2개(`Title_BGM.wav`, `Main_BGM.wav`)뿐이며, **61종의 효과음은 전부 코드로 합성된다**.

합성 기본기	용도
<code>Tone</code> / <code>Noise</code>	사인파 스위프, 시드 고정 잡음
<code>Bell</code>	배음별 감쇠 속도를 달리한 오르골·유리·마림바 음색
<code>LowPass</code> / <code>HighPass</code> / <code>BandPass</code> / <code>Resonant</code>	필터. 휘두르는 바람, 빔의 훑는 느낌
<code>Pluck</code> / <code>Shape</code> / <code>Swell</code> / <code>FadeOut</code>	포락선. 타격의 날카로움, 지속음의 부풀기
<code>Reverb</code> / <code>Tremolo</code> / <code>Normalize</code>	공간감과 최종 정리

난수는 `xorshift` 시드를 코드에 고정해, 어느 기기에서 실행해도 동일한 소리가 나도록 했다. AI에게는 소리를 "파일로 구해 오라"가 아니라 **"이런 인상의 소리를 이 재료들로 조립하라"**고 지시했고, 그 결과 아래처럼 각 스킬의 성격을 수치로 설명하는 코드가 만들어졌다.

```
/// 4 · Battle Roar — 화면 안의 모든 몬스터를 다섯 번 치는 마무리 기술입니다.
///
/// 눈에 보이는 피해가 다섯 번 들어가므로 소리도 **다섯 번 두드립니다.**
/// 한 번만 크게 울리면 화면과 소리의 핏수가 어긋나 얼어맞는 느낌이 사라집니다.
/// 뒤로 갈수록 조금씩 높고 작게 해서, 퍼져 나가며 찾아드는 충격파로 들리게 합니다.
private static float[] SkillBattleRoar()
{
    var mix = SfxSynth.Buffer(.95f);
    SfxSynth.Mix(mix, Swing(.26f, 260f, 1600f, 5971u, .7f), .6f); // 내지르는 준비 동작
    SfxSynth.Mix(mix, Rumble(.9f, 58f, 30f, 9f, 5972u), .5f);

    for (var i = 0; i < 5; i++) // 다섯 번의 타격
    {
        var start = .1f + i * .16f;
        var fade = 1f - i * .13f;
        var seed = (uint)(5973 + i * 3);
        SfxSynth.Mix(mix, Slam(.34f, 300f + i * 40f, 30f, seed, .92f), fade, start);
        SfxSynth.Mix(mix, Cut(.09f, seed + 1), .34f * fade, start);
        SfxSynth.Mix(mix, Debris(.28f, 8, seed + 2, 700f, 3400f), .34f * fade, start + .02f);
    }

    SfxSynth.Reverb(mix, .5f, 1.6f, .45f);
    return SfxSynth.Normalize(mix, .92f);
}
```

4.2 수치 밸런싱 — 값이 아니라 시뮬레이션

스킬 쿨타임·마나 소모 같은 수치를 조정할 때, AI에게 값을 "적당히" 정하게 하지 않고 전투 루프를 재현하는 시뮬레이터를 먼저 작성해 후보안을 계량 비교하도록 지시했다.

실제로 "스킬이 끊기지 않게 해 달라"는 요구를 처리할 때 산출한 지표는 다음과 같다.

쿨타임 조합(초)	빈 시간	최장 공백	마나 최저	1번 스킬 비중
3 / 5 / 7 / 8 / 9 (초기값)	26.70 %	1.86 s	고갈	—
0.9 / 3 / 4 / 5 / 9	0.00 %	0.00 s	4,947	36.2 %
1.1 / 2 / 2.5 / 3 / 8 (채택)	0.00 %	0.00 s	4,957	22.7 %

각 후보안은 시드를 바꿔 가며 20,000초 분량을 반복 시뮬레이션했고, 무작위 선택뿐 아니라 가장 불리한 선택을 유도하는 적대적 시나리오까지 돌려 "한 번도 비지 않음"을 확인한 뒤 적용했다.

4.3 로컬라이제이션

한국어·일본어·영어 3개 국어를 `Resources/Localization/Localization.csv` 한 장 (289행)에서 관리한다. 씬의 텍스트에는 `LocalizedText` 컴포넌트가 번역 키를 들고 있으며, `LocalizationBinder` 에디터 도구가 이 부착 작업을 일괄 처리한다.

5. AI 대상 주요 프롬프트 및 지시사항

5.1 상시 적용 지시 (코딩 규약)

매 작업마다 반복해 말하지 않도록, 아래 규칙을 상시 지시로 고정해 두었다.

지시	의도
주석은 한국어로 쓰고, " 무엇을 하는지 "가 아니라 " 왜 그렇게 했는지 "를 남길 것	코드를 읽으면 무엇을 하는지는 보인다. 나중에 지우면 안 되는 이유를 남기게 함
UI 레이아웃 요청은 썬만 수정하고 C# 스크립트를 만들지 말 것. 단 "클릭하면 ~된다" 같은 동작 요청은 스크립트까지 구현할 것	단순 배치 변경에 불필요한 스크립트가 늘어나는 것을 방지
게임 내 설명 텍스트는 15~25자 한 문장 으로. 두 문장 금지	모바일 UI에서 문장이 길면 잘리거나 폰트가 과도하게 줄어듦
번역은 CSV 한 장 에서만 관리. 썬에 직접 문자열을 넣지 말 것	같은 이름의 형제 오브젝트가 많아 경로만으로는 식별이 어려움
수치(밸런스) 변경은 근거 수치 를 제시한 뒤 적용할 것	감으로 조정하면 회귀가 반복됨
작업 후 Unity 콘솔 에러 를 직접 확인하고 결과를 사실대로 보고할 것	"완료했습니다"만 하고 실제로는 깨져 있는 상황 방지

5.2 작업 유형별 프롬프트 패턴

① 기능 추가 — 맥락과 기존 규칙을 함께 준다

"Skill5라는 캐릭터 애니메이션을 추가했어. 캐릭터 애니메이터에 넣어줬고 **그거를 기존처럼 연결해서** 5번째 스킬이 발동되게 해줘. `01_Prefabs/Effects` 안에 Row라는 스킬 프리팹을 넣어줬으니 5번째 스킬이 발동되면 저게 실행되면 되고, 저건 **모든 몬스터에게 5차례 피해를 주는 스킬**로 만들어줘."

"기존처럼"이라는 한 마디가 핵심이다. AI가 먼저 기존 스킬 1~4번의 배선 방식(애니메이터 파라미터, 애니메이션 이벤트, 데이터 에셋 구조)을 조사한 뒤 **같은 규칙을 따르도록** 유도한다. 새 방식을 발명하지 않게 하는 것이 목적이다.

② 판단 위임 — 근거를 요구한다

"현재 몬스터 피격 시 붉은색으로 노출되는데 그냥 하얀색으로 하면 어떨까? **이건 너의 추천대로 맞춰줘.**"

이 지시에 대해 AI는 색상 값을 바로 바꾸지 않고, 몬스터를 배치한 임시 썬에서 1.0 / 1.4 / 1.8 / 2.4 밝기를 나란히 렌더링해 비교한 뒤 **1.8을 근거와 함께 제시**했다. (순백 1.0은 이미 흰색인 재질에 아무 변화를 주지 못하고, 2.4는 명암이 날아가 실루엣만 남는다.)

③ 결함 지적 — 증상만 말하고 원인은 AI가 찾게 한다

"쿨타임이 다 넉넉하게 남아있는데 도대체 왜 T-Pose가 나오는거야? 절대 안 나오게 해줘"

"몬스터를 너무 자주 찾잖아. 내가 사냥하던 몬스터가 죽었을 때에만 다른 몬스터를 타겟팅해야지"

원인을 지정해 주지 않는 편이 결과가 좋았다. 원인을 짚어 주면 AI가 그 가설에 갇혀 표면만 고치는 경우가 있었기 때문이다.

④ 반려와 재지시 — 짧고 분명하게

"아직도 1번이 너무 많이 발동돼. 1번을 쿨타임 늘리고 나머지를 좀 더 줄여줘"

이 지시에 대해 AI는 **요청을 그대로 따르면 오히려 손이 비게 된다는** 점을 시뮬레이션 수치로 제시하고, 목적(1번 비중 축소)은 달성하되 방법을 바꾼 대안 (1번은 유지하고 2~5번을 단축)을 채택했다. **지시를 맹종하지 않고 목적을 달성하는 것을 바람직한 행동으로** 요구했다.

5.3 프롬프트 설계에서 얻은 원칙

- 목적을 말하고 수단은 열어 둔다. "이 값을 3으로 바꿔"보다 "쉬지 않고 이어지게 해줘"가 결과가 좋다.
- 기존 코드를 기준으로 지정한다. "기존처럼"이 새 규칙 발명을 막는다.
- 검증 방법까지 지시한다. "확인하고 알려줘"가 있으면 AI가 에디터에서 실제로 돌려 본다.
- 모르는 것은 모른다고 하게 한다. 확인하지 못한 항목은 보고에 명시하도록 요구했다.

6. 사례 연구 — AI 검증 루프가 잡아낸 결함

6.1 대기 자세가 T-Pose로 보이던 문제

증상 — 전투 중 스킬을 쓰지 않는 동안 캐릭터가 T자로 서 있음.

기존 코드의 대응 — 이전까지는 이 현상을 "전환 타이밍 문제"로 보고, 스킬이 끝난 뒤 0.5초를 강제로 쉬게 하는 코드가 들어가 있었다. 주석에도 그렇게 적혀 있었다.

AI가 확인한 실제 원인 — 에디터에서 애니메이터를 직접 열어 전 상태의 모션 참조를 출력한 결과:

```
CombatIdle -> MOTION IS NULL      ← 모션이 비어 있음
Run         -> Run.fbx           bones 23/23
Skill1      -> 1-1.fbx           bones 23/23
Skill2      -> 1-2.fbx           bones 23/23
...
```

.controller 파일은 애니메이션 클립을 fileId 라는 숫자로 참조하는데, FBX가 재임포트되면서 이 번호가 바뀌어 참조가 끊어져 있었다. 모션이 없는 상태는 Write Defaults 설정에 의해 모든 bones를 바인드 포즈(= T-Pose)로 되돌린다.

조치 — 올바른 클립을 다시 연결하고, 나아가 같은 사고가 재발하지 않도록 에디터 로드 시 모션이 빈 상태를 찾아 번호가 아닌 에셋 경로로 다시 연결하는 복구 루틴을 추가했다. 복구할 수 없으면 조용히 T-Pose가 되게 두지 않고 에러 로그로 알린다. 그리고 원인이 제거되었으므로 땀땀이던 0.5초 강제 대기를 제거했다.

시사점. 코드만 읽어서는 "0.5초 대기"가 왜 있는지 알 수 없고, 지우면 증상이 재발한다. 에디터의 직렬화된 실제 값을 조회할 수 있었기 때문에 근본 원인에 도달할 수 있었다.

6.2 포션 쿨타임 UI가 동작하지 않던 문제

증상 — 체력/마나 포션 아이콘의 원형 게이지(Fill)가 전혀 움직이지 않음.

AI가 씬의 컴포넌트 직렬화 값을 조회한 결과, 코드에는 문제가 없고 인스펙터 참조 4개 중 3개가 비어 있었다.

```
healthPotionCooldownFill = >>> 비어 있음 <<<
healthPotionCooldownText = Count
manaPotionCooldownFill   = >>> 비어 있음 <<<
manaPotionCooldownText   = >>> 비어 있음 <<<
```

씬에는 HPPosition/Fill, MPPosition/Fill 이 Radial 방식으로 정상 존재했으나 스크립트에 연결되지 않은 상태였다. AI가 계층을 탐색해 해당 컴포넌트를 찾아 연결하고 씬을 저장했다.

7. 외부 에셋 · 오픈소스 출처

7.1 게임 엔진 및 공식 패키지

이름	버전	비고
Unity	6000.3.13f1	Unity Companion License / 각 요금제 약관
Universal Render Pipeline	17.3.0	Unity 공식 패키지
Shader Graph / VFX Graph	17.3.0	Unity 공식 패키지
TextMesh Pro	내장	Unity 공식 패키지
Input System	1.19.0	Unity 공식 패키지
AI Navigation	2.0.12	Unity 공식 패키지
Post Processing	3.5.4	Unity 공식 패키지
Unity AI Assistant	2.17.0-pre.1	본 문서 3장의 MCP 브리지

7.2 백엔드 · 인증 SDK

이름	버전	용도 / 라이선스
Firebase Unity SDK (App / Auth / Firestore)	13.14.0	계정 인증 및 플레이어 데이터 저장 · Apache License 2.0
Google Sign-In for Unity	—	구글 계정 로그인 · Apache License 2.0
External Dependency Manager for Unity	—	네이티브 의존성 해석 · Apache License 2.0

Firebase 네이티브 바이너리는 GitHub 단일 파일 100MB 제한을 초과하는 항목 (`FirebaseCppApp-13_14_0.bundle` , 106MB)이 있어 **Git LFS**로 관리한다. LFS 대상은 `.gitattributes` 에 정의되어 있다.

7.3 아트 · VFX 에셋 (Unity Asset Store)

에셋	제작자	사용처
Suntail Village	Raygeas	전투 스테이지 배경 환경
Mummy Monster	amusedART	몬스터 모델 및 애니메이션
Modern UI Pack (v5.5.28)	Michsky	UI 컴포넌트 및 애니메이션
Casual RPG VFX / Environment VFX Pack	Lana Studio	스킬 및 환경 이펙트
Hyper Casual FX Vol.1 · Vol.2	Kyeoms (VFX_Klaus)	타격·획득 연출
Top Down Effects	SineVFX	스킬 범위 연출

Wooden Shield Pack	Korzanowski	장비(방패) 모델
--------------------	-------------	-----------

위 에셋은 모두 Unity Asset Store를 통해 취득했으며, 각 에셋의 **Asset Store EULA**를 따른다. 재배포 가능 형태가 아니므로 저장소에는 프로젝트 사용 목적으로만 포함되어 있다.

7.4 캐릭터 · 애니메이션

항목	출처 / 비고
플레이어 캐릭터 모델	프로젝트 자체 리소스 (<code>Assets/Pinous</code>)
캐릭터 애니메이션 클립	Mixamo 기반 클립 (클립 내부 식별자가 <code>mixamo.com</code>). Mixamo는 Adobe 계정 보유 시 상업적 사용이 허용된다.
UI 디자인 리소스	프로젝트 자체 제작 (<code>Assets/Pinous/03_Design</code>)

7.5 사운드

항목	출처
효과음 61종	전량 자체 생성 — SfxSynth 가 런타임 합성. 외부 음원 없음
BGM 2종	<code>Title_BGM.wav</code> , <code>Main_BGM.wav</code>

7.6 폰트

TextMesh Pro 폰트 에셋 4종(`Outline_KR` , `NoOutline_KR` , `Outline_JP` , `NoOutline_JP`)을 사용한다. 한국어와 일본어의 글리프 범위가 달라 폰트를 분리했으며, 일본어 문자는 `*_JP` 폰트 에셋에서만 정상 렌더링된다.

8. 정리

- AI를 편집기가 아니라 검증 주체로 세운 것이 이 프로젝트의 핵심 구조다. Unity MCP를 통해 AI가 자기 결과를 에디터에서 직접 확인함으로써, 코드만 봐서는 찾을 수 없는 **에셋·참조 계층의 결함**을 잡아낼 수 있었다.
- UI는 완성물을 편집하지 않고 **생성기(에디터 도구)**를 작성하게 하여, 화면 구성이 재실행 가능한 코드로 남았다.
- 밸런스 수치는 **시뮬레이션으로 계량 비교한 뒤** 적용했다.
- 지시는 **목적**을 말하고 **수단은 열어 두는** 형태가 가장 결과가 좋았으며, 요청을 그대로 따르면 목적을 해치는 경우 AI가 근거를 들어 대안을 제시하도록 요구했다.